

SQL Hands-on Practice (Executable Order)

Author: **Akshatha S**

SQL concepts including **DDL, DML, Aggregate Functions, Subqueries, Joins, and Window Functions**.

This notebook demonstrates SQL concepts including:

- DDL (Database Definition Language)
 - DML (Data Manipulation Language)
 - Aggregate Functions
 - Subqueries
 - Joins
 - Window Functions
-

1.Queries to Create DB ,Table with Constraints

1. Create Application Database

Syntax

```
CREATE DATABASE database_name;
```

Query 1

```
CREATE DATABASE app_db;
```

2. Use Application Database

Syntax

```
USE database_name;
```

Query 2

```
USE app_db;
```

3. Create Employee Table with Constraints

Syntax

```
CREATE TABLE table_name
(
column1 datatype CONSTRAINT,
column2 datatype CONSTRAINT,
...
);
```

Query 3

```
CREATE TABLE employee
(
emp_id VARCHAR(20) PRIMARY KEY,
emp_name VARCHAR(50) NOT NULL,
emp_salary INT DEFAULT 20000,
emp_dept VARCHAR(10),
emp_doj DATE NOT NULL,
emp_age INT CHECK(emp_age > 21)
);
```

4. Insert Multiple Employee Records

Syntax

```
INSERT INTO table_name VALUES (val1, val2, ...),
(val1, val2, ...);
```

Query 4

```
INSERT INTO employee VALUES
("ES001", "AMIT", 50000, "HR", "2019-12-03", 25),
("ES002", "BHARAT", 25000, "SALES", "2020-01-12", 30),
("ES003", "CHETAN", 40000, "IT", "2018-02-12", 22),
("ES004", "DIYA", 60000, "IT", "2017-03-02", 22),
("ES005", "JEEVAN", 80000, "HR", "2019-12-03", 25),
("ES006", "PRIYA", 75000, "SALES", "2020-01-12", 30),
("ES007", "JOY", 90000, "IT", "2018-02-12", 22),
("ES008", "RAVI", 45000, "FINANCE", "2018-02-03", 40),
("ES009", "TARUN", 65000, "FINANCE", "2012-02-03", 36);
```

5. View Employee Records

Syntax

```
SELECT * FROM table_name;
```

Query 5

```
SELECT * FROM employee;
```

6. Test Primary Key Constraint

Syntax

```
INSERT INTO table_name VALUES (duplicate_primary_key_value...);
```

Query 6

```
INSERT INTO employee VALUES  
("ES009", "ashwin", 65000, "FINANCE", "2012-02-03", 36);
```

7. Insert Valid Record

Query 7

```
INSERT INTO employee VALUES  
("ES010", "ashwin", 65000, "FINANCE", "2012-02-03", 36);
```

8. Test Age Constraint

Syntax

```
INSERT INTO table_name VALUES (invalid_constraint_value...);
```

Query 8

```
INSERT INTO employee VALUES  
("ES011", "drona", 65000, "FINANCE", "2012-02-03", 18);
```

9. Insert Values into Specific Columns

Syntax

```
INSERT INTO table_name(column1, column2...) VALUES (val1, val2...);
```

Query 9

```
INSERT INTO employee(emp_id,emp_name,emp_doj)
VALUES ("ES012","SHIVANI","2022-12-12");
```

10. Test NOT NULL Constraint

Syntax

```
INSERT INTO table_name(columns_without_not_null_column) VALUES (...);
```

Query 10

```
INSERT INTO employee(emp_id,emp_doj)
VALUES ("ES013","2025-12-12");
```

11. ALTER TABLE - Add Single Column

Syntax

```
ALTER TABLE table_name ADD column_name datatype;
```

Query 11

```
ALTER TABLE employee
ADD emp_gender VARCHAR(10);
```

12. ALTER TABLE - Add Multiple Columns

Syntax

```
ALTER TABLE table_name
ADD column1 datatype,
ADD column2 datatype;
```

Query 12

```
ALTER TABLE employee
ADD emp_status VARCHAR(15),
ADD emp_state VARCHAR(20);
```

13. ALTER TABLE - Drop Column

Syntax

```
ALTER TABLE table_name DROP COLUMN column_name;
```

Query 13

```
ALTER TABLE employee  
DROP COLUMN emp_status;
```

14. Update Records

Syntax

```
UPDATE table_name SET column=value WHERE condition;
```

Query 14

```
UPDATE employee  
SET emp_gender="NA"  
WHERE emp_gender IS NULL;
```

15. ALTER TABLE - Modify Column

Syntax

```
ALTER TABLE table_name MODIFY column_name new_datatype;
```

Query 15

```
ALTER TABLE employee  
MODIFY emp_gender VARCHAR(11) NOT NULL;
```

16. ALTER TABLE - Rename Column

Syntax

```
ALTER TABLE table_name  
CHANGE old_column_name new_column_name datatype;
```

Query 16

```
ALTER TABLE employee  
CHANGE emp_gender gender VARCHAR(11);
```

17. ALTER TABLE - Rename Table

Syntax

```
ALTER TABLE old_table_name RENAME TO new_table_name;
```

Query 17

```
ALTER TABLE employee  
RENAME TO employee_info;
```

2. QUERIES,SUBQUERIES & JOINS

Create Database datamites_db

Syntax

```
DROP DATABASE IF EXISTS database_name;
```

Query 1 – Drop database if it already exists

```
DROP DATABASE IF EXISTS datamites_db;
```

Syntax

```
CREATE DATABASE database_name;
```

Query 2 – Create a new database

```
CREATE DATABASE datamites_db;
```

Syntax

```
USE database_name;
```

Query 3 – Select the database to work with

```
USE datamites_db;
```

CREATE TABLES

Syntax

```
CREATE TABLE table_name(column datatype);
```

Query 4 – Create employee table

```
CREATE TABLE employee
(
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(30),
  dept_name VARCHAR(20),
  emp_salary INT
);
```

Syntax

```
CREATE TABLE table_name(column datatype);
```

Query 5 – Create department table

```
CREATE TABLE department (
  dept_id INT PRIMARY KEY,
  dept_name VARCHAR(30),
  dept_head VARCHAR(20)
);
```

Syntax

```
CREATE TABLE table_name(column datatype, FOREIGN KEY);
```

Query 6 – Create student table with foreign key

```
CREATE TABLE student (
  s_id INT PRIMARY KEY,
  s_name VARCHAR(30),
  s_age INT,
  dept_id INT,
  FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
```

INSERT DATA

Syntax

```
INSERT INTO table_name VALUES(values);
```

Query 7 – Insert multiple records into employee table

```
INSERT INTO employee VALUES
(101 , 'John', 'HR' , 10000),
(201,'Mary', 'Finance',20000),
(301,'Zara','IT',30000),
(401,'Rida','Sales',50000),
(501, 'Ram', 'HR',40000),
```

```
(601, 'Varun', 'IT', 70000),  
(701, 'Araav', 'HR', 60000),  
(801, 'Joy', 'IT', 90000),  
(302, 'Arun', 'Sales', 40000),  
(403, 'Joe', 'Finance', 35000),  
(704, "Dia", "Finance", 26000);
```

Syntax

```
INSERT INTO table_name VALUES(values);
```

Query 8 – Insert records into department table

```
INSERT INTO department VALUES  
(10, 'CDS', 'Ram Prasad'),  
(20, 'CDA', 'Nithish Kumar'),  
(30, 'CDE', 'Kishore Kamath'),  
(40, 'AIE', 'Bipin Roy'),  
(50, 'CS', 'Lohith Reddy'),  
(60, 'IOT', 'Sarvesh Yadav');
```

Syntax

```
INSERT INTO table_name VALUES(values);
```

Query 9 – Insert records into student table

```
INSERT INTO student VALUES  
(101, 'John', 30, 10),  
(102, 'Mary', 45, 20),  
(103, 'Ravi', 50, 20),  
(104, 'Anita', 40, 30),  
(105, 'Suresh', 35, NULL),  
(106, 'Neha', 28, 10),  
(107, 'Arjun', 16, 20),  
(108, 'Kiran', 32, 40),  
(109, 'Pooja', 28, 50),  
(110, 'Amit', 22, NULL);
```

RETRIEVE DATA

Syntax

```
SELECT * FROM table_name;
```

Query 10 – Retrieve all records from employee table

```
SELECT * FROM employee;
```

Query 11 – Retrieve all records from department table


```
SELECT * FROM department;
```

Query 12 – Retrieve all records from student table

```
SELECT * FROM student;
```

AGGREGATE FUNCTIONS

Syntax

```
SELECT aggregate_function(column) FROM table_name;
```

Query 13 – What is the maximum salary among all employees?

```
SELECT MAX(emp_salary) FROM employee;
```

Query 14 – What is the minimum salary among all employees?

```
SELECT MIN(emp_salary) FROM employee;
```

Query 15 – What is the average salary of employees?

```
SELECT AVG(emp_salary) FROM employee;
```

Query 16 – What is the total salary paid to all employees?

```
SELECT SUM(emp_salary) FROM employee;
```

Query 17 – How many employees are present in the table?

```
SELECT COUNT(emp_id) FROM employee;
```

GROUP BY

Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name;
```

Query 18 – How many employees are there in each department?

```
SELECT dept_name, COUNT(emp_id)
FROM employee
GROUP BY dept_name;
```

DISTINCT

Syntax

```
SELECT DISTINCT column_name FROM table_name;
```

Query 19 – What are the unique department names present in employee table?

```
SELECT DISTINCT dept_name FROM employee;
```

ORDER BY

Syntax

```
SELECT column_name FROM table_name ORDER BY column_name ASC | DESC;
```

Query 20 – How can we sort employee salaries in ascending order?

```
SELECT emp_name, emp_salary  
FROM employee  
ORDER BY emp_salary ASC;
```

Query 21 – How can we sort employee salaries in descending order?

```
SELECT emp_name, emp_salary  
FROM employee  
ORDER BY emp_salary DESC;
```

LIMIT

Syntax

```
SELECT * FROM table_name LIMIT number_of_rows;
```

Query 22 – How can we retrieve the first three employee records?

```
SELECT * FROM employee  
LIMIT 3;
```

Query 23 – How can we find the employee with the highest salary?

```
SELECT * FROM employee  
ORDER BY emp_salary DESC  
LIMIT 1;
```

WHERE CONDITIONS

Syntax

```
SELECT * FROM table_name WHERE condition;
```

Query 24 – How can we retrieve employees who belong to HR department?

```
SELECT * FROM employee  
WHERE dept_name = 'HR';
```

Query 25 – How can we find employees whose salary is greater than 50000?

```
SELECT * FROM employee  
WHERE emp_salary > 50000;
```

Query 26 – How can we find employees whose salary is between 20000 and 50000?

```
SELECT * FROM employee  
WHERE emp_salary BETWEEN 20000 AND 50000;
```

LIKE OPERATOR

Syntax

```
SELECT column_name FROM table_name WHERE column_name LIKE pattern;
```

Query 27 – Which employees have names starting with the letter 'A'?

```
SELECT emp_name FROM employee  
WHERE emp_name LIKE 'A%';
```

Query 28 – Which employees have names ending with the letter 'a'?

```
SELECT emp_name FROM employee  
WHERE emp_name LIKE '%a';
```

SUBQUERIES

Syntax

```
SELECT * FROM table_name WHERE column_name = (subquery);
```

Query 29 – Which employee earns the maximum salary in the company?

```
SELECT * FROM employee
WHERE emp_salary = (SELECT MAX(emp_salary) FROM employee);
```

Query 30 – Which employees earn more than the average salary?

```
SELECT * FROM employee
WHERE emp_salary > (SELECT AVG(emp_salary) FROM employee);
```

Query 31 – Who is the employee with the second highest salary?

```
SELECT * FROM employee
WHERE emp_salary < (SELECT MAX(emp_salary) FROM employee)
ORDER BY emp_salary DESC
LIMIT 1;
```

JOINS

Syntax

```
SELECT columns FROM table1 JOIN table2 ON condition;
```

Query 32 – How can we retrieve student names along with their department names?

```
SELECT s.s_name, d.dept_name
FROM student s
JOIN department d
ON s.dept_id = d.dept_id;
```

Syntax

```
SELECT columns FROM table1 LEFT JOIN table2 ON condition;
```

Query 33 – How can we retrieve all students even if they are not assigned to any department?

```
SELECT s.s_name, d.dept_name
FROM student s
LEFT JOIN department d
ON s.dept_id = d.dept_id;
```

Syntax

```
SELECT columns FROM table1 LEFT JOIN table2 ON condition WHERE condition;
```

Query 34 – Which students are not assigned to any department?

```
SELECT s.s_name  
FROM student s  
LEFT JOIN department d  
ON s.dept_id = d.dept_id  
WHERE d.dept_name IS NULL;
```

GROUP BY WITH JOIN

Syntax

```
SELECT column_name, COUNT(column)  
FROM table1 JOIN table2  
GROUP BY column_name;
```

Query 35 – How many students are there in each department?

```
SELECT d.dept_name, COUNT(s.s_id)  
FROM student s  
JOIN department d  
ON s.dept_id = d.dept_id  
GROUP BY d.dept_name;
```

HAVING

Syntax

```
SELECT column_name, COUNT(column)  
FROM table_name  
GROUP BY column_name  
HAVING condition;
```

Query 36 – Which departments have more than one student?

```
SELECT d.dept_name, COUNT(s.s_id)  
FROM student s  
JOIN department d  
ON s.dept_id = d.dept_id  
GROUP BY d.dept_name  
HAVING COUNT(s.s_id) > 1;
```

WINDOW FUNCTIONS

Syntax

```
SELECT column, COUNT() OVER(PARTITION BY column) FROM table_name;
```

Query 37 – How can we count students in each department using window function?

```
SELECT s.*,  
COUNT(*) OVER(PARTITION BY dept_id) AS dept_count  
FROM student s;
```

Syntax

```
SELECT column, RANK() OVER(PARTITION BY column ORDER BY column) FROM  
table_name;
```

Query 38 – How can we rank employees by salary within each department?

```
SELECT emp_name, dept_name, emp_salary,  
RANK() OVER(PARTITION BY dept_name ORDER BY emp_salary DESC) AS  
rank_in_dept  
FROM employee;
```

Syntax

```
SELECT column, RANK() OVER(ORDER BY column) FROM table_name;
```

Query 39 – How can we rank all employees based on salary across the company?

```
SELECT emp_name, emp_salary,  
RANK() OVER(ORDER BY emp_salary DESC) AS overall_rank  
FROM employee;
```

In []: